

课程作业 2

尧祥临 202518023406038 前沿交叉科学学院

1 任务介绍

本次作业的目标是构建并训练一个用于图像分类任务的深度学习模型，所用数据集包含 5 类花卉图像，分别为 daisy、dandelion、rose、sunflower 和 tulip，数据集已经分好成训练集和验证集以及测试集。具体任务包括模型设计与实现、训练过程分析以及结果展示。

2 模型架构

为了实现图像分类，我根据 ResNet 架构搭建了一个卷积神经网络模型。该模型共包含 4 个主要残差层，各层的残差块的数量分别为 3、4、6 和 3 个，每个残差块采用的是 ResNet34 架构中的基础块，它由两个 3×3 卷积层和一个残差连接构成。以下部分将分别介绍残差块的结构、整体网络架构、数据预处理与增强方法，以及模型的超参数设置与训练配置。

2.1 Residual Block

残差块是 ResNet 架构的核心组件，旨在解决深层网络中的梯度消失问题。每个残差块由两个连续的 3×3 卷积层构成：第一层卷积的步长由 `stride` 控制，在进行下采样时步长为 2，否则为 1；第二层卷积固定使用步长为 1。步长为 1 的时候能保持分辨率不变是因为两个卷积层均使用 `padding=1`，同理步长为 2 的时候分辨率会变成输入的一半。通道数由 `in_channels` 和 `out_channels` 指定。每个卷积层后面都跟有批归一化层和 ReLU 激活函数，并且在第二个批归一化层后、激活函数前进行残差连接。为了保证残差连接时参与相加的两路特征在维度上的一致性，当输入和输出的通道数不匹配或步长不为 1 时，会通过一个 1×1 卷积层和批归一化层对输入进行调整。在本模型中，`expansion` 被设置为 1，输出通道数不进行额外扩张，如果需要扩张可以调整该参数，例如在 ResNet50 中，`expansion` 就需要设置为 4。下面是残差块的具体实现代码：

```
1 class ResidualBlock(nn.Module):
2     expansion = 1 # ResNet34
3     def __init__(self, in_channels, out_channels, stride = 1):
4         super(ResidualBlock, self).__init__()
5         self.conv1 = nn.Conv2d(in_channels, out_channels,
6                                 kernel_size=3, stride=stride, padding=1, bias=False)
7         self.bn1 = nn.BatchNorm2d(out_channels)
8         self.conv2 = nn.Conv2d(out_channels, out_channels,
9                                 kernel_size=3, stride=1, padding=1, bias=False)
10        self.bn2 = nn.BatchNorm2d(out_channels)
11        self.relu = nn.ReLU(inplace=True)
12        self.shortcut = nn.Sequential() # 残差连接
13        if stride != 1 or in_channels != self.expansion * out_channels:
```

```

14         self.shortcut = nn.Sequential(
15             nn.Conv2d(in_channels, self.expansion * out_channels,
16                       kernel_size = 1, stride = stride, bias = False),
17             nn.BatchNorm2d(self.expansion * out_channels)
18         )
19
20     def forward(self, x):
21         out = self.conv1(x)
22         out = self.bn1(out)
23         out = self.relu(out)
24         out = self.conv2(out)
25         out = self.bn2(out)
26         out += self.shortcut(x) #  $F(x) + x$ 
27         out = self.relu(out)
28         return out

```

2.2 整体网络架构

基于上述残差块，这里构建了一个包含 4 个主要残差层的深度卷积神经网络。

网络的第一部分是一个初始的卷积层，使用`stride`为 2 和`padding`为 3 的 7×7 卷积核进行卷积操作，将输入图像的分辨率从 224×224 降至 112×112 ，随后接一个批归一化层和 ReLU 激活函数。这之后通过一个`stride`为 2 和`padding`为 1 的 3×3 Max 池化层进一步将分辨率降至 56×56 。

接下来是 4 个残差层，分别包含 3、4、6 和 3 个残差块。经过第一个残差层，图的分辨率保持为 56×56 ；经过第二个残差层时，图的分辨率降至 28×28 ；经过第三个残差层时，分辨率进一步降至 14×14 ；最后经过第四个残差层时，分辨率降至 7×7 。每个残差层的输出通道数依次为 64、128、256 和 512，所以最终输出的张量维度为 $512 \times 7 \times 7$ 。

在经过 4 个残差层之后，网络通过平均池化层进一步将张量维度降低到 $512 \times 1 \times 1$ ，并随后展平为 512 维的向量，最后通过一个全连接层的分类头映射到 5 个类别上，完成分类任务。

为了更完整的实践神经网络的搭建与训练过程，这里不使用预训练权重，而是初始化网络中的权重参数，如果是卷积层则使用 Kaiming 初始化，批归一化层的权重初始化为 1，偏置初始化为 0。下面是整体网络架构的具体实现代码：

```

1 class ResNet(nn.Module):
2     def __init__(self, block, layers, num_classes):
3         super(ResNet, self).__init__()
4         self.in_channels = 64
5         self.conv1 = nn.Conv2d(3, 64, kernel_size = 7, stride = 2, padding = 3, bias = False
6             ) #  $7 \times 7$  的初始卷积层,  $224 \rightarrow 112$ 
7         self.bn1 = nn.BatchNorm2d(64)
8         self.relu = nn.ReLU(inplace=True)
9         self.maxpool = nn.MaxPool2d(kernel_size = 3, stride = 2, padding = 1) # Max池化层,
10             112  $\rightarrow$  56
11         self.Layer1 = self._make_layer(block, 64, layers[0], stride = 1) # 第一层, 56  $\rightarrow$  56

```

```
10     self.Layer2 = self._make_layer(block, 128, layers[1], stride = 2) # 第二层, 56 -> 28
11     self.Layer3 = self._make_layer(block, 256, layers[2], stride = 2) # 第三层, 28 -> 14
12     self.Layer4 = self._make_layer(block, 512, layers[3], stride = 2) # 第四层, 14 -> 7
13     self.avgpool = nn.AdaptiveAvgPool2d((1, 1)) # 平均池化层, 7 -> 1
14     self.fc = nn.Linear(512 * block.expansion, num_classes) # 全连接层
15     self._initialize_weights()
16
17     def _make_layer(self, block, out_channels, blocks, stride = 1):
18         layers = []
19         layers.append(block(self.in_channels, out_channels, stride))
20         self.in_channels = out_channels * block.expansion # 更新输入通道数
21         for _ in range(1, blocks): # 添加剩余的残差块
22             layers.append(block(self.in_channels, out_channels))
23         return nn.Sequential(*layers)
24
25     def _initialize_weights(self):
26         for m in self.modules():
27             if isinstance(m, nn.Conv2d):
28                 nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
29             elif isinstance(m, nn.BatchNorm2d):
30                 nn.init.constant_(m.weight, 1)
31                 nn.init.constant_(m.bias, 0)
32
33     def forward(self, x):
34         x = self.conv1(x)
35         x = self.bn1(x)
36         x = self.relu(x)
37         x = self.maxpool(x)
38         x = self.Layer1(x)
39         x = self.Layer2(x)
40         x = self.Layer3(x)
41         x = self.Layer4(x)
42         x = self.avgpool(x)
43         x = torch.flatten(x, 1)
44         x = self.fc(x)
45         return x
```

2.3 数据预处理与增强

在数据预处理方面，考虑到提供的花卉图象具有不同的分辨率，所以这里我设计了一个`Sample`函数来读取对应数据，构造好样本，并把子文件夹名当作图片的标签。对于训练集，首先对读取到的图象进行统一的缩放，将较短边调整为 256 像素，然后随机裁剪出 224×224 的区域，随后进行了一系列经典的数据增强操作，包括随机水平翻转、随机旋转（角度范围为 ± 15 度）、颜色抖动（亮度、对比

度、饱和度和色调的调整范围分别为 0.2、0.2、0.2 和 0.1)。完成前面的增强步骤后将图象转换为张量并进行归一化处理，进行归一化的均值和标准差均采用 ImageNet 数据集的统计值是因为我认为这个值会比直接用 0.5 和 1.0 更合适一些，能更好地适应自然图象的分布。最后一步加入了随机擦除操作，擦除的概率为 0.1，擦除区域的比例范围为 0.02 到 0.33，更进一步的提升模型的泛化能力。对于验证集和测试集，则采用统一的缩放和中心裁剪来保证输入图象的一致性，除了不进行数据增强和随机擦除外，其他处理步骤与训练集相同。下面是数据预处理与增强的具体实现代码：

```
1 def Sample(data_path, reinforce = False, batch_size = 32):
2     IMAGENET_MEAN = (0.485, 0.456, 0.406) # ImageNet数据集的均值
3     IMAGENET_STD = (0.229, 0.224, 0.225) # ImageNet数据集的标准差
4     if reinforce: # 如果reinforce为True, 则进行数据增强
5         transform = transforms.Compose([
6             transforms.Resize(256),
7             transforms.RandomResizedCrop(224), # 随机裁剪224x224
8             transforms.RandomHorizontalFlip(), # 随机水平翻转
9             transforms.RandomRotation(15), # 随机旋转
10            transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1), #
                随机颜色抖动
11            transforms.ToTensor(),
12            transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD), # 归一化
13            transforms.RandomErasing(p=0.1, scale=(0.02, 0.33)) # 随机擦除
14        ])
15     else:
16         transform = transforms.Compose([
17             transforms.Resize(256),
18             transforms.CenterCrop(224),
19             transforms.ToTensor(),
20             transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)
21        ])
22     dataset = datasets.ImageFolder(root = data_path, transform = transform)
23     dataloader = torch.utils.data.DataLoader(dataset, batch_size = batch_size, shuffle =
        True)
24     return dataloader
```

2.4 模型超参数与训练设置

在模型的训练过程中，选择了多分类任务中常用的交叉熵作为损失函数。训练批次大小设置为 64，最大训练轮次设置为 100 个 epoch。初始学习率设置为 0.001，而在训练中使用CosineAnnealingLR余弦退火调节器来调整学习率，周期设置为最大训练轮次。优化器这里选择了 AdamW 优化器，权重衰减系数为 0.001，以帮助模型更好地收敛并防止过拟合。为了进一步防止过拟合，我设定了一个早停机制，耐心值为 15，即如果在连续 15 个 epoch 内验证集的准确率没有提升，则提前终止训练过程，而训练过程中会保存下验证集上准确率最高的模型的权重。为了更好的监控训练过程，我使用了 TensorBoard 来记录训练和验证的损失及准确率。下面是训练配置的具体实现代码：

```
1 def Train(num_classes, batch_size, lr, epochs, patience,
2           train_data_path, val_data_path, save_model_path, log_dir, device):
3     writer = SummaryWriter(log_dir) # TensorBoard日志记录器
4     model = ResNet(ResidualBlock, [3, 4, 6, 3], num_classes).to(device)
5     optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=0.001)
6     scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
7     train_loader = Sample(train_data_path, reinforce=True, batch_size = batch_size)
8     val_loader = Sample(val_data_path, batch_size = batch_size)
9     best_val_accuracy = 0
10    epochs_no_improve = 0
11    global_step = 0
12    for epoch in range(epochs):
13        running_loss = 0.0 # 记录训练集损失
14        correct_train = 0 # 记录训练集正确预测数
15        total_train = 0 # 记录训练集总数
16        for inputs, labels in train_loader:
17            inputs, labels = inputs.to(device), labels.to(device)
18            optimizer.zero_grad()
19            outputs = model(inputs)
20            loss = criterion(outputs, labels)
21            running_loss += loss.item() * inputs.size(0)
22            _, predicted_train = torch.max(outputs.data, 1)
23            total_train += labels.size(0)
24            correct_train += (predicted_train == labels).sum().item()
25            loss.backward()
26            optimizer.step()
27            global_step += 1
28        epoch_loss = running_loss / len(train_loader.dataset) # 计算epoch的平均损失
29        train_accuracy = 100 * correct_train / total_train
30        writer.add_scalar('Train/Loss_Epoch', epoch_loss, epoch)
31        writer.add_scalar('Train/Accuracy', train_accuracy, epoch)
32        val_accuracy, val_loss = Evaluate(model, val_loader, device) # 在验证集上评估
33        writer.add_scalar('Validation/Accuracy', val_accuracy, epoch)
34        writer.add_scalar('Validation/Loss', val_loss, epoch)
35        print(f"Epoch {epoch+1}, "
36              f"Train Loss: {epoch_loss:.4f}, "
37              f"Train Accuracy: {train_accuracy:.2f}%, "
38              f"Validation Loss: {val_loss:.4f}, "
39              f"Validation Accuracy: {val_accuracy:.2f}%")
40        scheduler.step() # 根据验证集准确率更新学习率
41        if val_accuracy > best_val_accuracy:
42            best_val_accuracy = val_accuracy
43            epochs_no_improve = 0
```

```
44     torch.save(model.state_dict(), save_model_path) # 保存最佳模型权重
45 else:
46     epochs_no_improve += 1
47     if epochs_no_improve >= patience:
48         break
49 writer.close() # 关闭TensorBoard记录器
50 model.load_state_dict(torch.load(save_model_path)) # 加载最佳模型权重
51 return model
52 ...
53 num_classes = 5
54 batch_size = 64
55 lr = 0.001 # 学习率
56 epochs = 100 # 最大训练轮数
57 patience = 15 # 早停耐心值
58 criterion = nn.CrossEntropyLoss() # 交叉熵作为损失函数
59 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

3 训练过程

在模型训练过程中，我使用了前面提到的配置和超参数设置。训练过程最终没有触发早停机制，模型在 100 个 epoch 后完成训练，模型的在验证集上达到的最高的准确率为 83.9%。图 1 所示的为训练和验证过程中损失值和准确率的变化趋势，可以看到训练集和验证集上的损失值均呈现下降趋势，而准确率则逐渐提升，表明模型在训练过程中有效的学习到了数据的特征。此外，可以看到验证集上的损失值和准确率在不部分时间都是优于训练集的，这可能得益于模型训练之前进行的数据增强操作，提升了模型的泛化能力。验证集上的准确率和损失值在第 80 轮后基本就趋于平稳，模型已经达到的较好的收敛的状态。值得注意的是，验证集上的准确率和损失值的波动相对训练集来说更为明显，这可能是由于验证集的样本量较小，导致评估结果受个别样本影响较大所致。

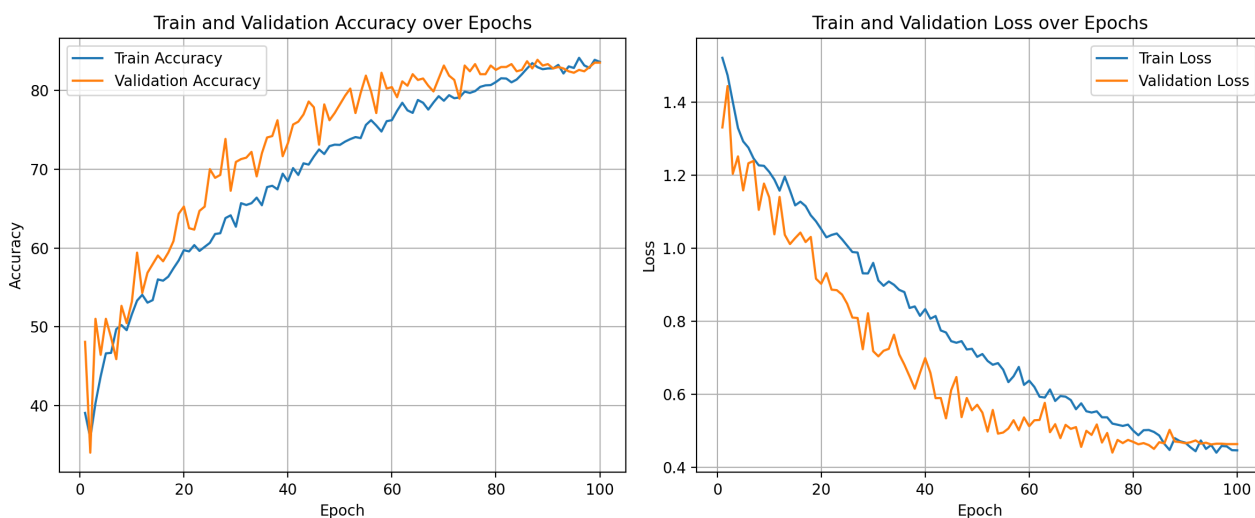


图 1: 训练和验证集上的损失值与准确率变化曲线

4 结果分析

在测试集上评估最终训练完成的模型，准确率达到 100%，不过这主要是因为测试集样本量太小，每个类别仅有 1 张图象，所以这一结果并不能充分反映模型的实际性能。图 2所示的为验证集上的混淆矩阵，图 3所示的为验证集上各类别的精确率、召回率与 F1 分数。可以看到模型在大部分类别上都表现良好，只有少数类别之间存在一定的混淆，例如 roses 类别有 20 张图象被错误分类为 tulips，从而导致 roses 类别的召回率仅为 0.663，而 tulips 类别的精确率也只有 0.766，两类的 F1 分数分别为 0.754 和 0.787。模型在 daisy 和 dandelion 类别上表现尤为出色，召回率均超过了 0.9，F1 分数也都在 0.85 以上。

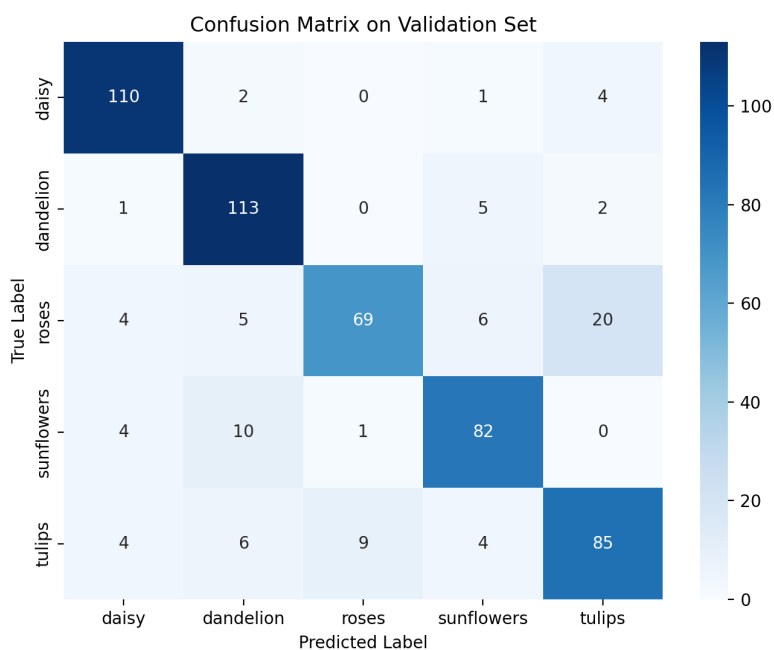


图 2: 验证集上的混淆矩阵

为了更好的展示模型实际上对于图片的预测情况，这里展示了模型对于测试集的部分图片的预测情况，图 4为模型对 daisy 和 tulips 的预测结果。可以看到模型是能够准确识别出这两张图片所对应的花卉类别的，但是模型对于两个图片预测的类别的概率分布存在较大的差异，对于 daisy 图片，如预测输出 1结果所示，模型对于 daisy 类别的预测概率高达 0.9999，几乎可以确定无疑地将其分类为 daisy，而对于其他类别的预测概率则非常低，均接近于 0。这表明模型对于 daisy 类别的识别非常自信。而对于 tulips 图片，预测输出 2结果所示，模型对于 tulips 类别的预测概率为 0.4321，虽然是最高的类别，但与其他类别（如 daisy 的 0.2995 和 dandelion 的 0.2015）之间的差距并不大，这表明模型在对 tulips 类别的识别上存在一定的不确定性，可能是因为图片中花卉的特征与其他类别存在一定的相似性，导致模型难以做出明确的区分。这再一次说明我的模型目前对于某些类型尤其是 tulips 的特征学习的还不够充分，未来可以尝试通过更多的数据增强、更复杂的模型架构或者更长时间的训练来提升模型的性能。

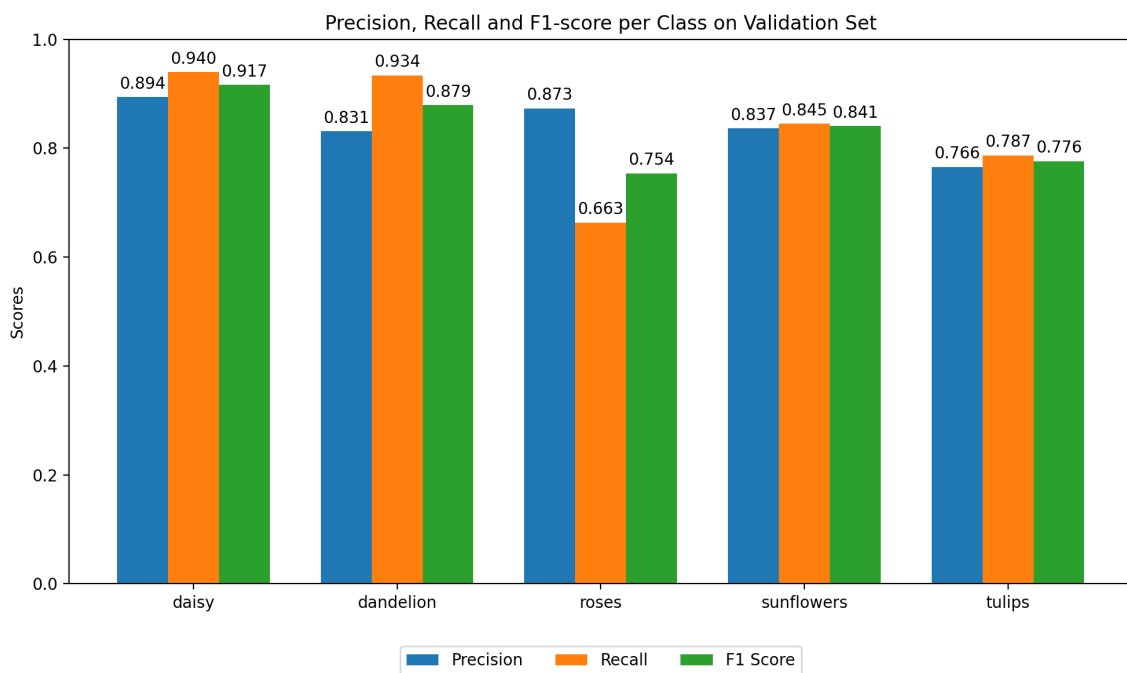


图 3: 各类别的精确率、召回率与 F1 分数

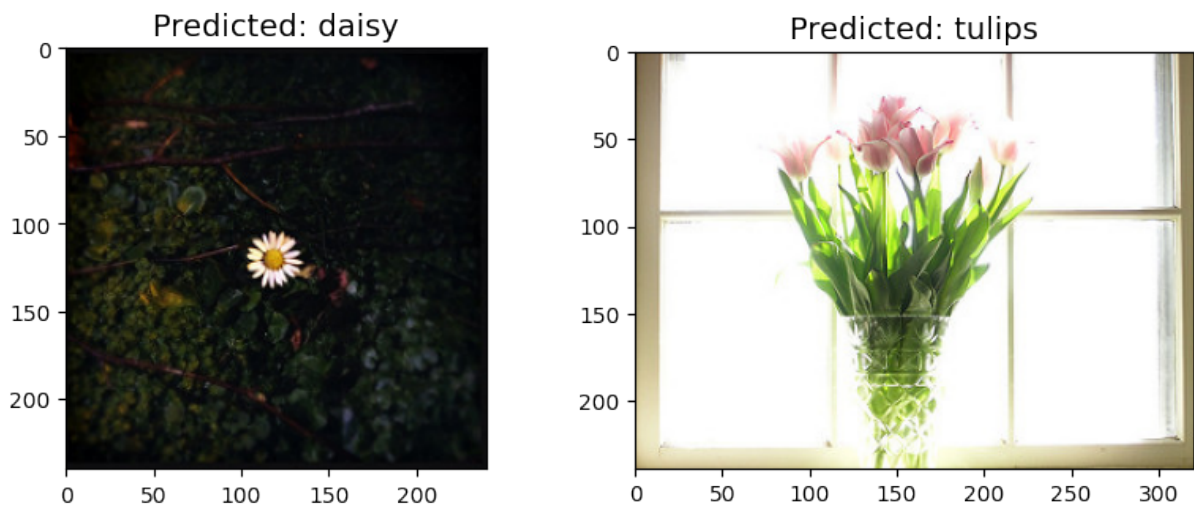


图 4: 模型对测试集图片的预测结果展示（左：daisy，右：tulips）

预测输出 1: Daisy prediction output

```
1 Predicted Class: daisy
2 Class Probabilities:
3     daisy : 0.9999 |#####
4     dandelion : 0.0001 |
5     roses : 0.0000 |
6     sunflowers : 0.0000 |
7     tulips : 0.0000 |
```

预测输出 2: Tulip prediction output

```
1 Predicted Class: tulips
2 Class Probabilities:
3     daisy : 0.2995 |#####
4     daisy : 0.2995 |#####
5     dandelion : 0.2015 |####
6     roses : 0.0633 |#
7     sunflowers : 0.0036 |
8     tulips : 0.4321 |#####
```

5 总结

本次作业中，我实现了一个基于 ResNet 架构的卷积神经网络模型来完成花卉图象分类任务，最终经过训练之后模型在验证集上达到了 83.9% 的准确率。尽管测试集上的准确率达到 100%，但由于测试集样本量较小，这一结果并不能充分反映模型的实际性能。经过具体分析，发现模型在某些类别上存在一定的混淆，尤其是 roses 和 tulips 类别之间，说明模型对这些类别的特征的学习还不够充分。